

These classes are provided in the [MediaImport sample](#) and can be added to your project without modifications. After adding the helper classes to your project, declare a class member variable of type **GeneratorIncrementalLoadingClass** that will be used later in this example.

C#

```
GeneratorIncrementalLoadingClass<ImportableItemWrapper> itemsToImport =  
null;
```

Find available sources from which media can be imported

In the click handler for the find sources button, call the static method [PhotoImportManager.FindAllSourcesAsync](#) to start the system searching for devices from which media can be imported. After awaiting the completion of the operation, loop through each [PhotoImportSource](#) object in the returned list and add an entry to the **ComboBox**, setting the **Tag** property to the source object itself so it can be easily retrieved when the user makes a selection.

C#

```
private async void findSourcesButton_Click(object sender, RoutedEventArgs e)  
{  
    var sources = await PhotoImportManager.FindAllSourcesAsync();  
    foreach (PhotoImportSource source in sources)  
    {  
        ComboBoxItem item = new ComboBoxItem();  
        item.Content = source.DisplayName;  
        item.Tag = source;  
        sourcesComboBox.Items.Add(item);  
    }  
}
```

Declare a class member variable to store the user's selected import source.

C#

```
PhotoImportSource importSource;
```

In the [SelectionChanged](#) handler for the import source **ComboBox**, set the class member variable to the selected source and then call the **FindItems** helper method which will be shown later in this article.

C#

```
private void sourcesComboBox_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    this.importSource = (PhotoImportSource)
    ((ComboBoxItem)sourcesComboBox.SelectedItem).Tag;
    FindItems();
}
```

Find items to import

Add class member variables of type [PhotoImportSession](#) and [PhotoImportFindItemsResult](#) which will be used in the following steps.

C#

```
PhotoImportSession importSession;
PhotoImportFindItemsResult itemsResult;
```

In the **FindItems** method, initialize the **CancellationTokenSource** variable so it can be used to cancel the find operation if necessary. Within a **try** block, create a new import session by calling [CreateImportSession](#) on the [PhotoImportSource](#) object selected by the user. Create a new [Progress](#) object to provide a callback to display the progress of the find operation. Next, call [FindItemsAsync](#) to start the find operation. Provide a [PhotoImportContentTypeFilter](#) value to specify whether photos, videos, or both should be returned. Provide a [PhotoImportItemSelectionMode](#) value to specify whether all, none, or only the new media items are returned with their [IsSelected](#) property set to true. This property is bound to a checkbox for each media item in our **ListBox** item template.

FindItemsAsync returns an [IAsyncOperationWithProgress](#). The extension method [AsTask](#) is used to create a task that can be awaited, can be cancelled with the cancellation token, and that reports progress using the supplied **Progress** object.

Next the data binding helper class, **GeneratorIncrementalLoadingClass** is initialized. **FindItemsAsync**, when it returns from being awaited, returns a [PhotoImportFindItemsResult](#) object. This object contains status information about the find operation, including the success of the operation and the count of different types of media items that were found. The **FoundItems** property contains a list of [PhotoImportItem](#) objects representing the found media items. The **GeneratorIncrementalLoadingClass** constructor takes as arguments the total count of items that will be loaded incrementally, and a function that generates new items to be

loaded as needed. In this case, the provided lambda expression creates a new instance of the **ImportableItemWrapper** which wraps **PhotoImportItem** and includes a thumbnail for each item. Once the incremental loading class has been initialized, set it to the **ItemsSource** property of the **ListView** control in the UI. Now, the found media items will be loaded incrementally and displayed in the list.

Next, the status information of the find operation are output. A typical app will display this information to the user in the UI, but this example simply outputs the information to the debug console. Finally, set the cancellation token to null because the operation is complete.

C#

```
private async void FindItems()
{
    this.cts = new CancellationTokenSource();

    try
    {
        this.importSession = this.importSource.CreateImportSession();

        // Progress handler for FindItemsAsync
        var progress = new Progress<uint>((result) =>
        {
            System.Diagnostics.Debug.WriteLine(String.Format("Found {0}
Files", result.ToString()));
        });

        this.itemsResult =
            await
this.importSession.FindItemsAsync(PhotoImportContentTypeFilter.ImagesAndVideos, PhotoImportItemSelectionMode.SelectAll)
                .AsTask(this.cts.Token, progress);

        // GeneratorIncrementalLoadingClass is used to incrementally load
        items in the ListView view including thumbnails
        this.itemsToImport = new
GeneratorIncrementalLoadingClass<ImportableItemWrapper>
(this.itemsResult.TotalCount,
    (int index) =>
    {
        return new
ImportableItemWrapper(this.itemsResult.FoundItems[index]);
    });

        // Set the items source for the ListView control
        this.fileListView.ItemsSource = this.itemsToImport;

        // Log the find results
        if (this.itemsResult != null)
        {
```

```

        var findResultProperties = new System.Text.StringBuilder();
        findResultProperties.AppendLine(String.Format("Photos\t\t\t : {0} \t\t Selected Photos\t\t: {1}", itemsResult.PhotosCount, itemsResult.SelectedPhotosCount));
        findResultProperties.AppendLine(String.Format("Videos\t\t\t : {0} \t\t Selected Videos\t\t: {1}", itemsResult.VideosCount, itemsResult.SelectedVideosCount));
        findResultProperties.AppendLine(String.Format("SideCars\t\t : {0} \t\t Selected Sidecars\t: {1}", itemsResult.SidecarsCount, itemsResult.SelectedSidecarsCount));
        findResultProperties.AppendLine(String.Format("Siblings\t\t\t : {0} \t\t Selected Siblings\t: {1} ", itemsResult.SiblingsCount, itemsResult.SelectedSiblingsCount));
        findResultProperties.AppendLine(String.Format("Total Items Items\t : {0} \t\t Selected TotalCount \t: {1}", itemsResult.TotalCount, itemsResult.SelectedTotalCount));

        System.Diagnostics.Debug.WriteLine(findResultProperties.ToString());
    }

    if (this.itemsResult.HasSucceeded)
    {
        // Update UI to indicate success
        System.Diagnostics.Debug.WriteLine("FindItemsAsync succeeded.");
    }
    else
    {
        // Update UI to indicate that the operation did not complete
        System.Diagnostics.Debug.WriteLine("FindItemsAsync did not succeed or was not completed.");
    }
}
catch (Exception ex)
{
    System.Diagnostics.Debug.WriteLine("Photo import find items operation failed. " + ex.Message);
}

this.cts = null;
}

```

Import media items

Before implementing the import operation, declare a [PhotoImportImportItemsResult](#) object to store the results of the import operation. This will be used later to delete media items that were successfully imported from the source.

```
private PhotoImportImportItemsResult importedResult;
```

Before starting the media import operation initialize the **CancellationTokenSource** variable and by setting the value of the **ProgressBar** control to 0.

If there are no selected items in the **ListView** control, then there is nothing to import. Otherwise, initialize a **Progress** object to provide a progress callback which updates the value of the progress bar control. Register a handler for the **ItemImported** event of the **PhotoImportFindItemsResult** returned by the find operation. This event will be raised whenever an item is imported and, in this example, outputs the name of each imported file to the debug console.

Call **ImportItemsAsync** to begin the import operation. Just as with the find operation, the **AsTask** extension method is used to convert the returned operation to a task that can be awaited, reports progress, and can be cancelled.

After the import operation is complete, the operation status can be obtained from the **PhotoImportImportItemsResult** object returned by **ImportItemsAsync**. This example outputs the status information to the debug console and then finally, sets the cancellation token to null.

C#

```
private async void importButton_Click(object sender, RoutedEventArgs e)
{
    cts = new CancellationTokenSource();
    progressBar.Value = 0;

    try
    {
        if (itemsResult.SelectedTotalCount <= 0)
        {
            System.Diagnostics.Debug.WriteLine("Nothing Selected for
Import.");
        }
        else
        {
            var progress = new Progress<PhotoImportProgress>((result) =>
            {
                progressBar.Value = result.ImportProgress;
            });

            this.itemsResult.ItemImported += async (s, a) =>
            {
                await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, ()
=>
                {
```

```

System.Diagnostics.Debug.WriteLine(String.Format("Imported: {0}",
a.ImportedItem.Name));
    });
};

// import items from the our list of selected items
this.importedResult = await
this.itemsResult.ImportItemsAsync().AsTask(cts.Token, progress);

if (importedResult != null)
{
    StringBuilder importedSummary = new StringBuilder();
    importedSummary.AppendLine(String.Format("Photos Imported
\t: {0} ", importedResult.PhotosCount));
    importedSummary.AppendLine(String.Format("Videos Imported
\t: {0} ", importedResult.VideosCount));
    importedSummary.AppendLine(String.Format("SideCars Imported
\t: {0} ", importedResult.SidecarsCount));
    importedSummary.AppendLine(String.Format("Siblings Imported
\t: {0} ", importedResult.SiblingsCount));
    importedSummary.AppendLine(String.Format("Total Items
Imported \t: {0} ", importedResult.TotalCount));
    importedSummary.AppendLine(String.Format("Total Bytes
Imported \t: {0} ", importedResult.TotalSizeInBytes));

System.Diagnostics.Debug.WriteLine(importedSummary.ToString());
}

if (!this.importedResult.HasSucceeded)
{
    System.Diagnostics.Debug.WriteLine("ImportItemsAsync did not
succeed or was not completed");
}
}
}
catch (Exception ex)
{
    System.Diagnostics.Debug.WriteLine("Files could not be imported. " +
"Exception: " + ex.ToString());
}

cts = null;
}

```

Delete imported items

To delete the successfully imported items from the source from which they were imported, first initialize the cancellation token so that the delete operation can be cancelled and set the progress bar value to 0. Make sure that the [PhotoImportImportItemsResult](#) returned from [ImportItemsAsync](#) is not null. If not,

once again create a [Progress](#) object to provide a progress callback for the delete operation. Call [DeleteImportedItemsFromSourceAsync](#) to start deleting the imported items. Use [AsTask](#) to convert the result to an awaitable task with progress and cancellation capabilities. After awaiting, the returned [PhotoImportDeleteImportedItemsFromSourceResult](#) object can be used to get and display status information about the delete operation.

C#

```
private async void deleteButton_Click(object sender, RoutedEventArgs e)
{
    cts = new CancellationTokenSource();
    progressBar.Value = 0;

    try
    {
        if (importedResult == null)
        {
            System.Diagnostics.Debug.WriteLine("Nothing was imported for deletion.");
        }
        else
        {
            var progress = new Progress<double>((result) =>
            {
                this.progressBar.Value = result;
            });

            PhotoImportDeleteImportedItemsFromSourceResult deleteResult =
            await
            this.importedResult.DeleteImportedItemsFromSourceAsync().AsTask(cts.Token,
            progress);

            if (deleteResult != null)
            {
                StringBuilder deletedResults = new StringBuilder();
                deletedResults.AppendLine(String.Format("Total Photos Deleted:\t{0} ", deleteResult.PhotosCount));
                deletedResults.AppendLine(String.Format("Total Videos Deleted:\t{0} ", deleteResult.VideosCount));
                deletedResults.AppendLine(String.Format("Total Sidecars Deleted:\t{0} ", deleteResult.SidecarsCount));
                deletedResults.AppendLine(String.Format("Total Siblings Deleted:\t{0} ", deleteResult.SiblingsCount));
                deletedResults.AppendLine(String.Format("Total Files Deleted:\t{0} ", deleteResult.TotalCount));
                deletedResults.AppendLine(String.Format("Total Bytes Deleted:\t{0} ", deleteResult.TotalSizeInBytes));

                System.Diagnostics.Debug.WriteLine(deletedResults.ToString());
            }
        }
    }
}
```

```
        if (!deleteResult.HasSucceeded)
        {
            System.Diagnostics.Debug.WriteLine("Delete operation did not
succeed or was not completed");
        }
    }

}

catch (Exception ex)
{
    System.Diagnostics.Debug.WriteLine("Files could not be Deleted." +
"Exception: " + ex.ToString());
}

// set the CancellationTokenSource to null when the work is complete.
cts = null;

}
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Camera-independent Flashlight

Article • 10/20/2022

This article shows how to access and use a device's lamp, if one is present. Lamp functionality is managed separately from the device's camera and camera flash functionality. In addition to acquiring a reference to the lamp and adjusting its settings, this article also shows you how to properly free up the lamp resource when it's not in use, and how to detect when the lamp's availability changes in case it is being used by another app.

Get the device's default lamp

To get a device's default lamp device, call [Lamp.GetDefaultAsync](#). The lamp APIs are found in the [Windows.Devices.Lights](#) namespace. Be sure to add a using directive for this namespace before attempting to access these APIs.

C#

```
using Windows.Devices.Lights;
```

C#

```
Lamp lamp;
```

C#

```
lamp = await Lamp.GetDefaultAsync();

if (lamp == null)
{
    ShowErrorMessage("No Lamp device found");
    return;
}
```

If the returned object is **null**, the **Lamp** API is unsupported on the device. Some devices may not support the **Lamp** API even if there is a lamp physically present on the device.

Get a specific lamp using the lamp selector string

Some devices may have more than one lamp. To obtain a list of lamps available on the device, get the device selector string by calling [GetDeviceSelector](#). This selector string can then be passed into [DeviceInformation.FindAllAsync](#). This method is used to enumerate many different kinds of devices and the selector string lets the method know to return only lamp devices. The [DeviceInformationCollection](#) object returned from [FindAllAsync](#) is a collection of [DeviceInformation](#) objects representing the lamps available on the device. Select one of the objects in the list and then pass the [Id](#) property to [Lamp.FromIdAsync](#) to get a reference to the requested lamp. This example uses the [GetFirstOrDefault](#) extension method from the [System.Linq](#) namespace to select the [DeviceInformation](#) object where the [EnclosureLocation.Panel](#) property has a value of [Back](#), which selects a lamp that is on the back of the device's enclosure, if one exists.

Note that the [DeviceInformation](#) APIs are found in the [Windows.Devices Enumeration](#) namespace.

C#

```
using Windows.Devices Enumeration;  
using System.Linq;
```

C#

```
string selectorString = Lamp.GetDeviceSelector();  
  
DeviceInformationCollection devices = await  
DeviceInformation.FindAllAsync(selectorString);  
  
DeviceInformation deviceInfo =  
    devices.FirstOrDefault(di => di.EnclosureLocation != null &&  
        di.EnclosureLocation.Panel ==  
Windows.Devices Enumeration.Panel.Back);  
  
if (deviceInfo == null)  
{  
    ShowErrorMessage("No Lamp device found");  
}  
  
lamp = await Lamp.FromIdAsync(deviceInfo.Id);
```

Adjust lamp settings

After you have an instance of the [Lamp](#) class, turn the lamp on by setting the [IsEnabled](#) property to [true](#).

C#

```
lamp.IsEnabled = true;
```

Turn the lamp off by setting the **IsEnabled** property to **false**.

C#

```
lamp.IsEnabled = false;
```

Some devices have lamps that support color values. Check if a lamp supports color by checking the **IsColorSettable** property. If this value is **true**, you can set the color of the lamp with the **Color** property.

C#

```
if (lamp.IsColorSettable)
{
    lamp.Color = Windows.UI.Colors.Blue;
}
```

Register to be notified if the lamp availability changes

Lamp access is granted to the most recent app to request access. So, if another app is launched and requests a lamp resource that your app is currently using, your app will no longer be able to control the lamp until the other app has released the resource. To receive a notification when the availability of the lamp changes, register a handler for the **Lamp.AvailabilityChanged** event.

C#

```
lamp = await Lamp.GetDefaultAsync();

if (lamp == null)
{
    ShowErrorMessage("No Lamp device found");
    return;
}

lamp.AvailabilityChanged += Lamp_AvailabilityChanged;
```

In the handler for the event, check the [LampAvailabilityChanged.IsAvailable](#) property to determine if the lamp is available. In this example, a toggle switch for turning the lamp on and off is enabled or disabled based on the lamp availability.

C#

```
private void Lamp_AvailabilityChanged(Lamp sender,
LampAvailabilityChangedEventArgs args)
{
    lampToggleSwitch.IsEnabled = args.IsAvailable;
}
```

Properly dispose of the lamp resource when not in use

When you are no longer using the lamp, you should disable it and call [Lamp.Close](#) to release the resource and allow other apps to access the lamp. This property is mapped to the **Dispose** method if you are using C#. If you registered for the [AvailabilityChanged](#), you should unregister the handler when you dispose of the lamp resource. The right place in your code to dispose of the lamp resource depends on your app. To scope lamp access to a single page, release the resource in the [OnNavigatingFrom](#) event.

C#

```
protected override void OnNavigatingFrom(NavigatingCancelEventArgs e)
{
    lamp.AvailabilityChanged -= Lamp_AvailabilityChanged;
    lamp.IsEnabled = false;
    lamp.Dispose();
    lamp = null;
}
```

Related topics

- [Media playback](#)

Supported codecs

Article • 11/15/2024

This article lists the audio, video, and image codec and format availability for UWP apps by default for each device family. Note that these tables list the codecs that are included with the Windows 10 installation for the specified device family. Users and apps can install additional codecs that may be available to use. You can query at runtime for the set of codecs that are currently available for a specific device. For more information, see [Query for codecs installed on a device](#).

In the tables below "D" indicates decoder support and "E" indicates encoder support.

Audio codec & format support

The following tables show the audio codec and format support for each device family.

Note

Where AMR-NB support is indicated, this codec is not supported on Server SKUs.

ⓘ **Note**

On versions of Windows prior to Windows 11, version 24H2, the AC-3 codec is included. Beginning with Windows 11, version 24H2, the AC-3 codec is no longer included with Windows. However, many device manufacturers will pre-install an AC-3 codec.

Desktop

Expand table

[illegible]

Codec/Container	MPEG-4	MPEG-3	MPEG-2	ADTS	ASF	RIFF	AVI	AC-3	AMR	3GP	FLAC	WAV
MP3	D	D					D					
MPEG-1/2			D									
MS ADPCM												D
WMA 1/2/3					D/E							
WMA Pro					D/E							
WMA Voice					D/E							

Xbox

 Expand table

Codec/Container	MPEG-4	MPEG-3	MPEG-2	ADTS	ASF	RIFF	AVI	AC-3	AMR	3GP	FLAC	WAV
HE-AAC v1 / AAC+	D			D								
HE-AAC v2 / eAAC+	D			D								
AAC-LC	D/E			D								
AC3	D		D		D	D	D	D				
ALAC	D											
AMR-NB	D								D/E	D		
FLAC											D	
G.711 (A-Law, μ -law)												D
GSM 6.10												D
IMA ADPCM												D
LPCM												D/E
MP3	D	D					D					
MPEG-1/2			D									
MS ADPCM												D
WMA 1/2/3					D							
WMA Pro					D							
WMA Voice					D							

Video codec & format support

The following tables show the video codec and format support for each device family.

ⓘ Note

Where H.265 support is indicated, it is not necessarily supported by all devices within the device family.

Desktop

 Expand table

Codec/Container	fMP4	MPEG-4	MPEG-2 PS	MPEG-2 TS	MPEG-1	3GPP	3GPP2	AVCHD	ASF	AVI	MKV	DV
MPEG-1 [*]			D/E	D/E	D			D/E			D	
MPEG-2 [*]			D/E	D/E				D/E			D	
MPEG-4 (Part 2)		D				D				D	D	
H.265 ^{**}	D	D/E									D	
H.264	D/E	D/E	D/E	D/E		D/E		D/E		D	D	
H.263		D/E				D/E				D		
VC-1		D							D	D	D	
WMV7/8/9		D/E							D/E	D/E		
WMV9 Screen		D							D	D		
DV		D								D		D
VP8		D/E									D	
VP9	D	D/E									D	
AV1 ^{**}	D	D									D	
Motion JPEG		D								D		

^{*} MPEG-2/MPEG-1 codec is available with install of optional Microsoft UWP DVD application.

^{**} H.265 and AV1 are available with the install of the corresponding optional codec pack.

Mobile

 Expand table

Codec/Container	fMP4	MPEG-4	MPEG-2 PS	MPEG-2 TS	3GPP	AVCHD	ASF	AVI	MKV
MPEG-4 (Part 2)		D			D			D	D
H.265	D	D/E							D
H.264	D/E	D/E	D/E	D/E	D/E	D/E		D	D
H.263		D/E			D/E			D	D
VC-1		D						D	D
Motion JPEG		D						D	

Windows 10 IoT

Expand table

[illegible]

* MPEG-2/MPEG-1 codec is available with install of optional Microsoft UWP DVD application.

** H.265 and AV1 are available with the install of the corresponding optional codec pack.

Xbox

 Expand table

Codec/Container	fMP4	MPEG-4	MPEG-2 PS	MPEG-2 TS	MPEG-1	3GPP	AVCHD	ASF	AVI	MKV	DV
MPEG-1			D/E	D/E	D		D/E			D	
MPEG-2			D/E	D/E			D/E			D	
MPEG-4 (Part 2)		D				D			D	D	
H.265	D	D/E								D	
H.264	D/E	D/E	D/E	D/E		D/E	D/E		D	D	
H.263		D/E				D/E			D		
VC-1		D							D	D	
VP8		D/E								D	
VP9	D	D/E								D	
AV1	D	D								D	
DV											D
Motion JPEG		D							D		

Image codec & format support

 Expand table

Codec	Desktop	Other device families
BMP	D/E	D/E
DDS	D/E ¹	D/E ¹
DNG	D ²	D ²
GIF	D/E	D/E
ICO	D	D
JPEG	D/E	D/E

Codec	Desktop	Other device families
JPEG-XR	D/E	D/E
PNG	D/E	D/E
TIFF	D/E	D/E
Camera RAW	D ³	No

¹ DDS images using BC1 through BC5 compression are supported.

² DNG images with a non-RAW embedded preview are supported.

³ Only certain camera RAW formats are supported.

For more information on image codecs, see [Native WIC Codecs](#).

Timed text format support

Windows has native support for the following formats for timed text. For more information on implementing timed text, see [Add external timed text with TimedTextSource](#), [TimedTextSource Class](#), [IMFTimedText interface](#).

- [TTML \(Timed Text Markup Language\)](#) 
- [WebVTT \(Web Video Text Tracks\)](#) 
- [SubRip](#) 
- [Substation Alpha](#) 
- [VobSub](#) 
- [PGS \(Presentation Graphics Stream\)](#) 

Feedback

Was this page helpful?



[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Query for codecs installed on a device

Article • 10/20/2022

The [CodecQuery](#) class allows you to query for codecs installed on the current device. The list of codecs that are included with Windows 10 for different device families are listed in the article [Supported codecs](#), but since users and apps can install additional codecs on a device, you may want to query for codec support at runtime to determine what codecs are available on the current device.

The CodecQuery API is a member of the [Windows.Media.Core](#) namespace, so you will need to include this namespace in your app.

C#

```
using Windows.Media.Core;
```

Initialize a new instance of the **CodecQuery** class by calling the constructor.

C#

```
var codecQuery = new CodecQuery();
```

The [FindAllAsync](#) method returns all installed codecs that match the supplied parameters. These parameters include a [CodecKind](#) value specifying whether you are querying for audio or video codecs or both, a [CodecCategory](#) value specifying whether you are querying for encoders or decoders, and a string that represents the media encoding subtype for which you are querying, such as H.264 video or MP3 audio.

Specify empty string for the subtype value to return codecs for all subtypes. The following example lists all of the video encoders installed on the device.

C#

```
ReadOnlyList<CodecInfo> result =  
    await codecQuery.FindAllAsync(CodecKind.Video, CodecCategory.Encoder,  
    "");  
  
foreach (var codecInfo in result)  
{  
    this.codecResultsTextBox.Text +=  
        "=====\n";  
    this.codecResultsTextBox.Text += string.Format("Codec: {0}\n",  
        codecInfo.DisplayName);  
    this.codecResultsTextBox.Text += string.Format("Kind: {0}\n",  
        codecInfo.Kind.ToString());  
}
```

```

        this.codecResultsTextBox.Text += string.Format("Category: {0}\n",
        codecInfo.Category.ToString());
        this.codecResultsTextBox.Text += string.Format("Trusted: {0}\n",
        codecInfo.IsTrusted.ToString());

        foreach (string subType in codecInfo.Subtypes)
        {
            this.codecResultsTextBox.Text += string.Format("    Subtype: {0}\n",
            subType);
        }
    }
}

```

The subtype string you pass into **FindAllAsync** can either be a string representation of the subtype GUID which is defined by the system or a FOURCC code for the subtype. The set of supported media subtype GUIDs are listed in the articles [Audio Subtype GUIDs](#) and [Video Subtype GUIDs](#), but the [CodecSubtypes](#) class provides properties that return the GUID values for each supported subtype. For more information on FOURCC codes, see [FOURCC Codes](#)

The following example specifies the FOURCC code "H264" to determine if there is an H.264 video decoder installed on the device. You could perform this query before attempting to play back H.264 video content. You can also handle unsupported codecs at playback time. For more information, see [Handle unsupported codecs and unknown errors when opening media items](#).

C#

```

IReadOnlyList<CodecInfo> h264Result = await
    codecQuery.FindAllAsync(CodecKind.Video, CodecCategory.Decoder, "H264");

if (h264Result.Count > 0)
{
    this.codecResultsTextBox.Text = "H264 decoder is present.";
}

```

The following example queries to determine if a FLAC audio encoder is installed on the current device and, if so, a [MediaEncodingProfile](#) is created for the subtype which could be used for capturing audio to a file or transcoding audio from another format to a FLAC audio file.

C#

```

IReadOnlyList<CodecInfo> flacResult =
    await codecQuery.FindAllAsync(CodecKind.Audio, CodecCategory.Encoder,
    CodecSubtypes.AudioFormatFlac);

if (flacResult.Count > 0)

```

```
{
    AudioEncodingProperties audioProps = new AudioEncodingProperties();
    audioProps.Subtype = CodecSubtypes.AudioFormatFlac;
    audioProps.SampleRate = 44100;
    audioProps.ChannelCount = 2;
    audioProps.Bitrate = 128000;
    audioProps.BitsPerSample = 32;

    MediaEncodingProfile encodingProfile = new MediaEncodingProfile();
    encodingProfile.Audio = audioProps;
    encodingProfile.Video = null;
}
```

Related topics

- [Media playback](#)
- [Basic photo, video, and audio capture with MediaCapture](#)
- [Transcode media files](#)
- [Supported codecs](#)

Screen capture

Article • 10/18/2023

Starting in Windows 10, version 1803, the [Windows.Graphics.Capture](#) namespace provides APIs to acquire frames from a display or application window, to create video streams or snapshots to build collaborative and interactive experiences.

With screen capture, developers invoke secure system UI for end users to pick the display or application window to be captured, and a yellow notification border is drawn by the system around the actively captured item. In the case of multiple simultaneous capture sessions, a yellow border is drawn around each item being captured.

ⓘ Note

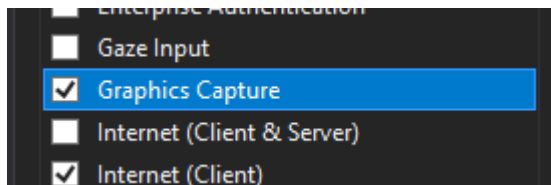
The screen capture APIs are only supported on Windows devices and Windows Mixed Reality immersive headsets.

This article describes capturing a single image of the display or application window. For information on encoding frames captured from the screen to a video file, see [Screen capture to video](#)

Add the screen capture capability

The APIs found in the **Windows.Graphics.Capture** namespace require a general capability to be declared in your application's manifest:

1. Open **Package.appxmanifest** in the **Solution Explorer**.
2. Select the **Capabilities** tab.
3. Check **Graphics Capture**.



Launch the system UI to start screen capture

Before launching the system UI, you can check to see if your application is currently able to take screen captures. There are several reasons why your application might not be able to use screen capture, including if the device does not meet hardware requirements

or if the application targeted for capture blocks screen capture. Use the **IsSupported** method in the [GraphicsCaptureSession](#) class to determine if UWP screen capture is supported:

C#

```
// This runs when the application starts.
public void OnInitialization()
{
    if (!GraphicsCaptureSession.IsSupported())
    {
        // Hide the capture UI if screen capture is not supported.
        CaptureButton.Visibility = Visibility.Collapsed;
    }
}
```

Once you've verified that screen capture is supported, use the [GraphicsCapturePicker](#) class to invoke the system picker UI. The end user uses this UI to select the display or application window of which to take screen captures. The picker will return a [GraphicsCaptureItem](#) that will be used to create a **GraphicsCaptureSession**:

C#

```
public async Task StartCaptureAsync()
{
    // The GraphicsCapturePicker follows the same pattern the
    // file pickers do.
    var picker = new GraphicsCapturePicker();
    GraphicsCaptureItem item = await picker.PickSingleItemAsync();

    // The item may be null if the user dismissed the
    // control without making a selection or hit Cancel.
    if (item != null)
    {
        // We'll define this method later in the document.
        StartCaptureInternal(item);
    }
}
```

Because this is UI code, it needs to be called on the UI thread. If you're calling it from the code-behind for a page of your application (like **MainPage.xaml.cs**) this is done for you automatically, but if not, you can force it to run on the UI thread with the following code:

C#

```
CoreWindow window = CoreApplication.MainView.CoreWindow;
```

```
await window.Dispatcher.RunAsync(CoreDispatcherPriority.Normal, async () =>
{
    await StartCaptureAsync();
});
```

Create a capture frame pool and capture session

Using the **GraphicsCaptureItem**, you will create a **Direct3D11CaptureFramePool** with your D3D device, supported pixel format (**DXGI_FORMAT_B8G8R8A8_UNORM**), number of desired frames (which can be any integer), and frame size. The **ContentSize** property of the **GraphicsCaptureItem** class can be used as the size of your frame:

ⓘ Note

On systems with Windows HD color enabled, the content pixel format might not necessarily be **DXGI_FORMAT_B8G8R8A8_UNORM**. To avoid pixel overclipping (i.e. the captured content looks washed out) when capturing HDR content, consider using **DXGI_FORMAT_R16G16B16A16_FLOAT** for every component in the capturing pipeline, including the **Direct3D11CaptureFramePool**, the target destination such as **CanvasBitmap** [↗](#). Depends on the need, additional processing such as saving to HDR content format or HDR-to-SDR tone mapping might be required. This article will focus on SDR content capturing. For more information, please see [Using DirectX with high dynamic range Displays and Advanced Color](#).

C#

```
private GraphicsCaptureItem _item;
private Direct3D11CaptureFramePool _framePool;
private CanvasDevice _canvasDevice;
private GraphicsCaptureSession _session;

public void StartCaptureInternal(GraphicsCaptureItem item)
{
    _item = item;

    _framePool = Direct3D11CaptureFramePool.Create(
        _canvasDevice, // D3D device
        DirectXPixelFormat.B8G8R8A8UIntNormalized, // Pixel format
        2, // Number of frames
        _item.Size); // Size of the buffers
}
```

Next, get an instance of the **GraphicsCaptureSession** class for your **Direct3D11CaptureFramePool** by passing the **GraphicsCaptureItem** to the **CreateCaptureSession** method:

C#

```
_session = _framePool.CreateCaptureSession(_item);
```

Once the user has explicitly given consent to capturing an application window or display in the system UI, the **GraphicsCaptureItem** can be associated to multiple **CaptureSession** objects. This way your application can choose to capture the same item for various experiences.

To capture multiple items at the same time, your application must create a capture session for each item to be captured, which requires invoking the picker UI for each item that is to be captured.

Acquire capture frames

With your frame pool and capture session created, call the **StartCapture** method on your **GraphicsCaptureSession** instance to notify the system to start sending capture frames to your app:

C#

```
_session.StartCapture();
```

To acquire these capture frames, which are [Direct3D11CaptureFrame](#) objects, you can use the **Direct3D11CaptureFramePool.FrameArrived** event:

C#

```
_framePool.FrameArrived += (s, a) =>
{
    // The FrameArrived event fires for every frame on the thread that
    // created the Direct3D11CaptureFramePool. This means we don't have to
    // do a null-check here, as we know we're the only one
    // dequeuing frames in our application.

    // NOTE: Disposing the frame retires it and returns
    // the buffer to the pool.
    using (var frame = _framePool.TryGetNextFrame())
    {
        // We'll define this method later in the document.
        ProcessFrame(frame);
    }
}
```

```
}  
};
```

It is recommended to avoid using the UI thread if possible for **FrameArrived**, as this event will be raised every time a new frame is available, which will be frequent. If you do choose to listen to **FrameArrived** on the UI thread, be mindful of how much work you're doing every time the event fires.

Alternatively, you can manually pull frames with the **Direct3D11CaptureFramePool.TryGetNextFrame** method until you get all of the frames that you need.

The **Direct3D11CaptureFrame** object contains the properties **ContentSize**, **Surface**, and **SystemRelativeTime**. The **SystemRelativeTime** is QPC ([QueryPerformanceCounter](#)) time that can be used to synchronize other media elements.

Process capture frames

Each frame from the **Direct3D11CaptureFramePool** is checked out when calling **TryGetNextFrame**, and checked back in according to the lifetime of the **Direct3D11CaptureFrame** object. For native applications, releasing the **Direct3D11CaptureFrame** object is enough to check the frame back in to the frame pool. For managed applications, it is recommended to use the **Direct3D11CaptureFrame.Dispose** (**Close** in C++) method. **Direct3D11CaptureFrame** implements the [IClosable](#) interface, which is projected as [IDisposable](#) for C# callers.

Applications should not save references to **Direct3D11CaptureFrame** objects, nor should they save references to the underlying Direct3D surface after the frame has been checked back in.

While processing a frame, it is recommended that applications take the [ID3D11Multithread](#) lock on the same device that is associated with the **Direct3D11CaptureFramePool** object.

The underlying Direct3D surface will always be the size specified when creating (or recreating) the **Direct3D11CaptureFramePool**. If content is larger than the frame, the contents are clipped to the size of the frame. If the content is smaller than the frame, then the rest of the frame contains undefined data. It is recommended that applications copy out a sub-rect using the **ContentSize** property for that **Direct3D11CaptureFrame** to avoid showing undefined content.

Take a screenshot

In our example, we convert each **Direct3D11CaptureFrame** into a [CanvasBitmap](#), which is part of the [Win2D APIs](#).

C#

```
// Convert our D3D11 surface into a Win2D object.
CanvasBitmap canvasBitmap = CanvasBitmap.CreateFromDirect3D11Surface(
    _canvasDevice,
    frame.Surface);
```

Once we have the **CanvasBitmap**, we can save it as an image file. In the following example, we save it as a PNG file in the user's **Saved Pictures** folder.

C#

```
StorageFolder pictureFolder = KnownFolders.SavedPictures;
StorageFile file = await pictureFolder.CreateFileAsync("test.png",
    CreationCollisionOption.ReplaceExisting);

using (var fileStream = await file.OpenAsync(FileAccessMode.ReadWrite))
{
    await canvasBitmap.SaveAsync(fileStream, CanvasBitmapFileFormat.Png,
    1f);
}
```

React to capture item resizing or device lost

During the capture process, applications may wish to change aspects of their **Direct3D11CaptureFramePool**. This includes providing a new Direct3D device, changing the size of the frame buffers, or even changing the number of buffers within the pool. In each of these scenarios, the **Recreate** method on the **Direct3D11CaptureFramePool** object is the recommended tool.

When **Recreate** is called, all existing frames are discarded. This is to prevent handing out frames whose underlying Direct3D surfaces belong to a device that the application may no longer have access to. For this reason, it may be wise to process all pending frames before calling **Recreate**.

Putting it all together

The following code snippet is an end-to-end example of how to implement screen capture in a UWP application. In this sample, we have two buttons in the front-end: one calls **Button_ClickAsync**, and the other calls **ScreenshotButton_ClickAsync**.

ⓘ Note

This snippet uses [Win2D](#), a library for 2D graphics rendering. See their documentation for information about how to set it up for your project.

C#

```
using Microsoft.Graphics.Canvas;
using Microsoft.Graphics.Canvas.UI.Composition;
using System;
using System.Numerics;
using System.Threading.Tasks;
using Windows.Foundation;
using Windows.Graphics;
using Windows.Graphics.Capture;
using Windows.Graphics.DirectX;
using Windows.Storage;
using Windows.UI;
using Windows.UI.Composition;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Hosting;

namespace ScreenCaptureTest
{
    /// <summary>
    /// An empty page that can be used on its own or navigated to within a
    Frame.
    /// </summary>
    public sealed partial class MainPage : Page
    {
        // Capture API objects.
        private SizeInt32 _lastSize;
        private GraphicsCaptureItem _item;
        private Direct3D11CaptureFramePool _framePool;
        private GraphicsCaptureSession _session;

        // Non-API related members.
        private CanvasDevice _canvasDevice;
        private CompositionGraphicsDevice _compositionGraphicsDevice;
        private Compositor _compositor;
        private CompositionDrawingSurface _surface;
        private CanvasBitmap _currentFrame;
        private string _screenshotFilename = "test.png";

        public MainPage()
        {
```

```

        this.InitializeComponent();
        Setup();
    }

    private void Setup()
    {
        _canvasDevice = new CanvasDevice();

        _compositionGraphicsDevice =
CanvasComposition.CreateCompositionGraphicsDevice(
            Window.Current.Compositor,
            _canvasDevice);

        _compositor = Window.Current.Compositor;

        _surface = _compositionGraphicsDevice.CreateDrawingSurface(
            new Size(400, 400),
            DirectXPixelFormat.B8G8R8A8UIntNormalized,
            DirectXAlphaMode.Premultiplied);    // This is the only
value that currently works with                // the composition APIs.

        var visual = _compositor.CreateSpriteVisual();
        visual.RelativeSizeAdjustment = Vector2.One;
        var brush = _compositor.CreateSurfaceBrush(_surface);
        brush.HorizontalAlignmentRatio = 0.5f;
        brush.VerticalAlignmentRatio = 0.5f;
        brush.Stretch = CompositionStretch.Uniform;
        visual.Brush = brush;
        ElementCompositionPreview.SetElementChildVisual(this, visual);
    }

    public async Task StartCaptureAsync()
    {
        // The GraphicsCapturePicker follows the same pattern the
        // file pickers do.
        var picker = new GraphicsCapturePicker();
        GraphicsCaptureItem item = await picker.PickSingleItemAsync();

        // The item may be null if the user dismissed the
        // control without making a selection or hit Cancel.
        if (item != null)
        {
            StartCaptureInternal(item);
        }
    }

    private void StartCaptureInternal(GraphicsCaptureItem item)
    {
        // Stop the previous capture if we had one.
        StopCapture();

        _item = item;
        _lastSize = _item.Size;
    }

```

```

        _framePool = Direct3D11CaptureFramePool.Create(
            _canvasDevice, // D3D device
            DirectXPixelFormat.B8G8R8A8UIntNormalized, // Pixel format
            2, // Number of frames
            _item.Size); // Size of the buffers

        _framePool.FrameArrived += (s, a) =>
        {
            // The FrameArrived event is raised for every frame on the
            // thread that created the Direct3D11CaptureFramePool. This means
            // we don't have to do a null-check here, as we know we're the
            // only one dequeuing frames in our application.

            // NOTE: Disposing the frame retires it and returns
            // the buffer to the pool.

            using (var frame = _framePool.TryGetNextFrame())
            {
                ProcessFrame(frame);
            }
        };

        _item.Closed += (s, a) =>
        {
            StopCapture();
        };

        _session = _framePool.CreateCaptureSession(_item);
        _session.StartCapture();
    }

    public void StopCapture()
    {
        _session?.Dispose();
        _framePool?.Dispose();
        _item = null;
        _session = null;
        _framePool = null;
    }

    private void ProcessFrame(Direct3D11CaptureFrame frame)
    {
        // Resize and device-lost leverage the same function on the
        // Direct3D11CaptureFramePool. Refactoring it this way avoids
        // throwing in the catch block below (device creation could
        // always fail) along with ensuring that resize completes successfully
        // and isn't vulnerable to device-lost.
        bool needsReset = false;
        bool recreateDevice = false;

```

```

        if ((frame.ContentSize.Width != _lastSize.Width) ||
            (frame.ContentSize.Height != _lastSize.Height))
        {
            needsReset = true;
            _lastSize = frame.ContentSize;
        }

        try
        {
            // Take the D3D11 surface and draw it into a
            // Composition surface.

            // Convert our D3D11 surface into a Win2D object.
            CanvasBitmap canvasBitmap =
CanvasBitmap.CreateFromDirect3D11Surface(
                _canvasDevice,
                frame.Surface);

            _currentFrame = canvasBitmap;

            // Helper that handles the drawing for us.
            FillSurfaceWithBitmap(canvasBitmap);
        }

        // This is the device-lost convention for Win2D.
        catch (Exception e) when (_canvasDevice.IsDeviceLost(e.HResult))
        {
            // We lost our graphics device. Recreate it and reset
            // our Direct3D11CaptureFramePool.
            needsReset = true;
            recreateDevice = true;
        }

        if (needsReset)
        {
            ResetFramePool(frame.ContentSize, recreateDevice);
        }
    }

    private void FillSurfaceWithBitmap(CanvasBitmap canvasBitmap)
    {
        CanvasComposition.Resize(_surface, canvasBitmap.Size);

        using (var session =
CanvasComposition.CreateDrawingSession(_surface))
        {
            session.Clear(Colors.Transparent);
            session.DrawImage(canvasBitmap);
        }
    }

    private void ResetFramePool(SizeInt32 size, bool recreateDevice)
    {
        do
        {

```

```

        try
        {
            if (recreateDevice)
            {
                _canvasDevice = new CanvasDevice();

                _framePool.Recreate(
                    _canvasDevice,
                    DirectXPixelFormat.B8G8R8A8UIntNormalized,
                    2,
                    size);
            }
            // This is the device-lost convention for Win2D.
            catch (Exception e) when
(_canvasDevice.IsDeviceLost(e.HResult))
            {
                _canvasDevice = null;
                recreateDevice = true;
            }
        } while (_canvasDevice == null);
    }

    private async void Button_ClickAsync(object sender, RoutedEventArgs
e)
    {
        await StartCaptureAsync();
    }

    private async void ScreenshotButton_ClickAsync(object sender,
RoutedEventArgs e)
    {
        await SaveImageAsync(_screenshotFilename, _currentFrame);
    }

    private async Task SaveImageAsync(string filename, CanvasBitmap
frame)
    {
        StorageFolder pictureFolder = KnownFolders.SavedPictures;

        StorageFile file = await pictureFolder.CreateFileAsync(
            filename,
            CreationCollisionOption.ReplaceExisting);

        using (var fileStream = await
file.OpenAsync(FileAccessMode.ReadWrite))
        {
            await frame.SaveAsync(fileStream,
CanvasBitmapFileFormat.Png, 1f);
        }
    }
}

```

Record a video

If you want to record a video of your application, you can follow the walkthrough presented in the article [Screen capture to video](#). Or, you can use [Windows.Media.AppRecording namespace](#). This is part of the Desktop extension SDK, so it only works on Windows desktops and requires that you add a reference to it from your project. See [Programming with extension SDKs](#) for more information.

See also

- [Windows.Graphics.Capture Namespace](#)
- [Screen capture to video](#)

Screen capture to video

Article • 10/20/2022

This article describes how to encode frames captured from the screen with the `Windows.Graphics.Capture` APIs to a video file. For information on screen capturing still images, see [Screen capture](#). For a simple end-to-end sample app that utilizes the concepts and techniques shown in this article, see [SimpleRecorder](#).

Overview of the video capture process

This article provides a walkthrough of an example app that records the contents of a window to a video file. While it may seem like there is a lot of code required to implement this scenario, the high-level structure of a screen recorder app is fairly simple. The screen capture process uses three primary UWP features:

- The [Windows.Graphics.Capture](#) APIs do the work of actually grabbing the pixels from the screen. The [GraphicsCaptureItem](#) class represents the window or display being captured. [GraphicsCaptureSession](#) is used to start and stop the capture operation. The [Direct3D11CaptureFramePool](#) class maintains a buffer of frames into which the screen contents are copied.
- The [MediaStreamSource](#) class receives the captured frames and generates a video stream.
- The [MediaTranscoder](#) class receives the stream produced by the [MediaStreamSource](#) and encodes it into a video file.

The example code shown in this article can be categorized into a few different tasks:

- **Initialization** - This includes configuring the UWP classes described above, initializing the graphics device interfaces, picking a window to capture, and setting up the encoding parameters such as resolution and frame rate.
- **Event handlers and threading** - The primary driver of the main capture loop is the [MediaStreamSource](#) which requests frames periodically through the [SampleRequested](#) event. This example uses events to coordinate the requests for new frames between the different components of the example. Synchronization is important to allow frames to be captured and encoded simultaneously.
- **Copying frames** - Frames are copied from the capture frame buffer into a separate Direct3D surface that can be passed to the [MediaStreamSource](#) so that the resource isn't overwritten while being encoded. Direct3D APIs are used to perform this copy operation quickly.

About the Direct3D APIs

As stated above, the copying of each captured frame is probably the most complex part of the implementation shown in this article. At a low level, this operation is done using Direct3D. For this example, we are using the [SharpDX](#) library to perform the Direct3D operations from C#. This library is no longer officially supported, but it was chosen because its performance at low-level copy operations is well-suited for this scenario. We have tried to keep the Direct3D operations as discrete as possible to make it easier for you to substitute your own code or other libraries for these tasks.

Setting up your project

The example code in this walkthrough was created using the **Blank App (Universal Windows) C#** project template in Visual Studio 2019. In order to use the **Windows.Graphics.Capture** APIs in your app, you must include the **Graphics Capture** capability in the Package.appxmanifest file for your project. This example saves generated video files to the Videos Library on the device. To access this folder you must include the **Videos Library** capability.

To install the SharpDX Nuget package, in Visual Studio select **Manage Nuget Packages**. In the Browse tab search for the "SharpDX.Direct3D11" package and click **Install**.

Note that in order to reduce the size of the code listings in this article, the code in the walkthrough below omits explicit namespace references and the declaration of MainPage class member variables which are named with a leading underscore, "_".

Setup for encoding

The **SetupEncoding** method described in this section initializes some of the main objects that will be used to capture and encode video frames and sets up the encoding parameters for captured video. This method could be called programmatically or in response to a user interaction like a button click. The code listing for **SetupEncoding** is shown below after the descriptions of the initialization steps.

- **Check for capture support.** Before beginning the capture process, you need to call [GraphicsCaptureSession.IsSupported](#) to make sure that the screen capture feature is supported on the current device.
- **Initialize Direct3D interfaces.** This sample uses Direct3D to copy the pixels captured from the screen into a texture that is encoded as a video frame. The

helper methods used to initialize the Direct3D interfaces, **CreateD3DDevice** and **CreateSharpDXDevice**, are shown later in this article.

- **Initialize a GraphicsCaptureItem.** A [GraphicsCaptureItem](#) represents an item on the screen that is going to be captured, either a window or the entire screen. Allow the user to pick an item to capture by creating a [GraphicsCapturePicker](#) and calling [PickSingleItemAsync](#).
- **Create a composition texture.** Create a texture resource and an associated render target view that will be used to copy each video frame. This texture can't be created until the **GraphicsCaptureItem** has been created and we know its dimensions. See the description of the **WaitForNewFrame** to see how this composition texture is used. The helper method for creating this texture is also shown later in this article.
- **Create a MediaEncodingProfile and VideoStreamDescriptor.** An instance of the [MediaStreamSource](#) class will take images captured from the screen and encode them into a video stream. Then, the video stream will be transcoded into a video file by the [MediaTranscoder](#) class. A [VideoStreamDescriptor](#) provides encoding parameters, such as resolution and frame rate, for the **MediaStreamSource**. The video file encoding parameters for the **MediaTranscoder** are specified with a [MediaEncodingProfile](#). Note that the size used for video encoding doesn't have to be the same as the size of the window being captured, but to keep this example simple, the encoding settings are hard-coded to use the capture item's actual dimensions.
- **Create the MediaStreamSource and MediaTranscoder objects.** As mentioned above, the **MediaStreamSource** object encodes individual frames into a video stream. Call the constructor for this class, passing in the **MediaEncodingProfile** created in the previous step. Set the buffer time to zero and register handlers for the [Starting](#) and [SampleRequested](#) events, which will be shown later in this article. Next, construct a new instance of the **MediaTranscoder** class and enable hardware acceleration.
- **Create an output file** The final step in this method is to create a file to which the video will be transcoded. For this example, we will just create a uniquely named file in the Videos Library folder on the device. Note that in order to access this folder, your app must specify the "Videos Library" capability in the app manifest. Once the file has been created, open it for read and write, and pass the resulting stream into the **EncodeAsync** method which will be shown next.

```

private async Task SetupEncoding()
{
    if (!GraphicsCaptureSession.IsSupported())
    {
        // Show message to user that screen capture is unsupported
        return;
    }

    // Create the D3D device and SharpDX device
    if (_device == null)
    {
        _device = Direct3D11Helpers.CreateD3DDevice();
    }
    if (_sharpDxD3dDevice == null)
    {
        _sharpDxD3dDevice = Direct3D11Helpers.CreateSharpDXDevice(_device);
    }

    try
    {
        // Let the user pick an item to capture
        var picker = new GraphicsCapturePicker();
        _captureItem = await picker.PickSingleItemAsync();
        if (_captureItem == null)
        {
            return;
        }

        // Initialize a blank texture and render target view for copying
        // frames, using the same size as the capture item
        _composeTexture =
        Direct3D11Helpers.InitializeComposeTexture(_sharpDxD3dDevice,
        _captureItem.Size);
        _composeRenderTargetView = new
        SharpDX.Direct3D11.RenderTargetView(_sharpDxD3dDevice, _composeTexture);

        // This example encodes video using the item's actual size.
        var width = (uint)_captureItem.Size.Width;
        var height = (uint)_captureItem.Size.Height;

        // Make sure the dimensions are even. Required by some encoders.
        width = (width % 2 == 0) ? width : width + 1;
        height = (height % 2 == 0) ? height : height + 1;

        var temp =
        MediaEncodingProfile.CreateMp4(VideoEncodingQuality.HD1080p);
        var bitrate = temp.Video.Bitrate;
        uint framerate = 30;

        _encodingProfile = new MediaEncodingProfile();
        _encodingProfile.Container.Subtype = "MPEG4";
    }
}

```

```

        _encodingProfile.Video.Subtype = "H264";
        _encodingProfile.Video.Width = width;
        _encodingProfile.Video.Height = height;
        _encodingProfile.Video.Bitrates = bitrate;
        _encodingProfile.Video.FrameRate.Numerator = framerate;
        _encodingProfile.Video.FrameRate.Denominator = 1;
        _encodingProfile.Video.PixelAspectRatio.Numerator = 1;
        _encodingProfile.Video.PixelAspectRatio.Denominator = 1;

        var videoProperties =
VideoEncodingProperties.CreateUncompressed(MediaEncodingSubtypes.Bgra8,
width, height);
        _videoDescriptor = new VideoStreamDescriptor(videoProperties);

        // Create our MediaStreamSource
        _mediaStreamSource = new MediaStreamSource(_videoDescriptor);
        _mediaStreamSource.BufferTime = TimeSpan.FromSeconds(0);
        _mediaStreamSource.Starting += OnMediaStreamSourceStarting;
        _mediaStreamSource.SampleRequested +=
OnMediaStreamSourceSampleRequested;

        // Create our transcoder
        _transcoder = new MediaTranscoder();
        _transcoder.HardwareAccelerationEnabled = true;

        // Create a destination file - Access to the VideosLibrary requires
the "Videos Library" capability
        var folder = KnownFolders.VideosLibrary;
        var name = DateTime.Now.ToString("yyyyMMdd-HHmm-ss");
        var file = await folder.CreateFileAsync($"{name}.mp4");

        using (var stream = await file.OpenAsync(FileAccessMode.ReadWrite))

            await EncodeAsync(stream);
    }
    catch (Exception ex)
    {

        return;
    }
}

```

Start encoding

Now that the main objects have been initialized the **EncodeAsync** method is implemented to kick off the capture operation. This method first checks to make sure we aren't already recording, and if not, it calls the helper method **StartCapture** to begin capturing frames from the screen. This method is shown later in this article. Next, [PrepareMediaStreamSourceTranscodeAsync](#) is called to get the **MediaTranscoder** ready

to transcode the video stream produced by the **MediaStreamSource** object to the output file stream, using the encoding profile we created in the previous section. Once the transcoder has been prepared, call [TranscodeAsync](#) to start transcoding. For more information on using the **MediaTranscoder**, see [Transcode media files](#).

C#

```
private async Task EncodeAsync(IRandomAccessStream stream)
{
    if (!_isRecording)
    {
        _isRecording = true;

        StartCapture();

        var transcode = await
        _transcoder.PrepareMediaStreamSourceTranscodeAsync(_mediaStreamSource,
        stream, _encodingProfile);

        await transcode.TranscodeAsync();
    }
}
```

Handle MediaStreamSource events

The **MediaStreamSource** object takes frames that we capture from the screen and transforms them into a video stream that can be saved to a file using the **MediaTranscoder**. We pass the frames to the **MediaStreamSource** via handlers for the object's events.

The [SampleRequested](#) event is raised when the **MediaStreamSource** is ready for a new video frame. After making sure we are currently recording, the helper method **WaitForNewFrame** is called to get a new frame captured from the screen. This method, shown later in this article, returns a [ID3D11Surface](#) object containing the captured frame. For this example, we wrap the **IDirect3DSurface** interface in a helper class that also stores the system time at which the frame was captured. Both the frame and the system time are passed into the [MediaStreamSample.CreateFromDirect3D11Surface](#) factory method and the resulting [MediaStreamSample](#) is set to the [MediaStreamSourceSampleRequest.Sample](#) property of the [MediaStreamSourceSampleRequestedEventArgs](#). This is how the captured frame is provided to the **MediaStreamSource**.

C#

```

private void OnMediaStreamSourceSampleRequested(MediaStreamSource sender,
MediaStreamSourceSampleRequestedEventArgs args)
{
    if (_isRecording && !_closed)
    {
        try
        {
            using (var frame = WaitForNewFrame())
            {
                if (frame == null)
                {
                    args.Request.Sample = null;
                    Stop();
                    Cleanup();
                    return;
                }

                var timeStamp = frame.SystemRelativeTime;

                var sample =
MediaStreamSample.CreateFromDirect3D11Surface(frame.Surface, timeStamp);
                args.Request.Sample = sample;
            }
        }
        catch (Exception e)
        {
            Debug.WriteLine(e.Message);
            Debug.WriteLine(e.StackTrace);
            Debug.WriteLine(e);
            args.Request.Sample = null;
            Stop();
            Cleanup();
        }
    }
    else
    {
        args.Request.Sample = null;
        Stop();
        Cleanup();
    }
}

```

In the handler for the [Starting](#) event, we call **WaitForNewFrame**, but only pass the system time the frame was captured to the [MediaStreamSourceStartingRequest.SetActualStartPosition](#) method, which the **MediaStreamSource** uses to properly encode the timing of the subsequent frames.

C#

```

private void OnMediaStreamSourceStarting(MediaStreamSource sender,
MediaStreamSourceStartingEventArgs args)

```

```
{
    using (var frame = WaitForNewFrame())
    {
        args.Request.SetActualStartPosition(frame.SystemRelativeTime);
    }
}
```

Start capturing

The **StartCapture** method shown in this step is called from the **EncodeAsync** helper method shown in a previous step. First, this method initializes up a set of event objects that are used to control the flow of the capture operation.

- **_multithread** is a helper class wrapping the SharpDX library's **Multithread** object that will be used to make sure that no other threads access the SharpDX texture while it's being copied.
- **_frameEvent** is used to signal that a new frame has been captured and can be passed to the **MediaStreamSource**
- **_closedEvent** signals that recording has stopped and that we shouldn't wait for any new frames.

The frame event and closed event are added to an array so we can wait for either one of them in the capture loop.

The rest of the **StartCapture** method sets up the Windows.Graphics.Capture APIs that will do the actual screen capturing. First, an event is registered for the **CaptureItem.Closed** event. Next, a **Direct3D11CaptureFramePool** is created, which allows multiple captured frames to be buffered at a time. The **CreateFreeThreaded** method is used to create the frame pool so that the **FrameArrived** event is called on the pool's own worker thread rather than on the app's main thread. Next, a handler is registered for the **FrameArrived** event. Finally, a **GraphicsCaptureSession** is created for the selected **CaptureItem** and the capture of frames is initiated by calling **StartCapture**.

C#

```
public void StartCapture()
{
    _multithread =
        _sharpDxD3dDevice.QueryInterface<SharpDX.Direct3D11.Multithread>();
    _multithread.SetMultithreadProtected(true);
    _frameEvent = new ManualResetEvent(false);
    _closedEvent = new ManualResetEvent(false);
    _events = new[] { _closedEvent, _frameEvent };
}
```

```

_captureItem.Closed += OnClosed;
_framePool = Direct3D11CaptureFramePool.CreateFreeThreaded(
    _device,
    DirectXPixelFormat.B8G8R8A8UIntNormalized,
    1,
    _captureItem.Size);
_framePool.FrameArrived += OnFrameArrived;
_session = _framePool.CreateCaptureSession(_captureItem);
_session.StartCapture();
}

```

Handle graphics capture events

In the previous step we registered two handlers for graphics capture events and set up some events to help manage the flow of the capture loop.

The **FrameArrived** event is raised when the **Direct3D11CaptureFramePool** has a new captured frame available. In the handler for this event, call [TryGetNextFrame](#) on the sender to get the next captured frame. After the frame is retrieved, we set the **_frameEvent** so that our capture loop knows there is a new frame available.

C#

```

private void OnFrameArrived(Direct3D11CaptureFramePool sender, object args)
{
    _currentFrame = sender.TryGetNextFrame();
    _frameEvent.Set();
}

```

In the **Closed** event handler, we signal the **_closedEvent** so that the capture loop will know when to stop.

C#

```

private void OnClosed(GraphicsCaptureItem sender, object args)
{
    _closedEvent.Set();
}

```

Wait for new frames

The **WaitForNewFrame** helper method described in this section is where the heavy-lifting of the capture loop occurs. Remember, this method is called from the **OnMediaStreamSourceSampleRequested** event handler whenever the

MediaStreamSource is ready for a new frame to be added to the video stream. At a high-level, this function simply copies each screen-captured video frame from one Direct3D surface to another so that it can be passed into the **MediaStreamSource** for encoding while a new frame is being captured. This example uses the SharpDX library to perform the actual copy operation.

Before waiting for a new frame, the method disposes of any previous frame stored in the class variable, **_currentFrame**, and resets the **_frameEvent**. Then the method waits for either the **_frameEvent** or the **_closedEvent** to be signaled. If the closed event is set, then the app calls a helper method to cleanup the capture resources. This method is shown later in this article.

If the frame event is set, then we know that the **FrameArrived** event handler defined in the previous step has been called, and we begin the process of copying the captured frame data into a Direct3D 11 surface that will be passed to the **MediaStreamSource**.

This example uses a helper class, **SurfaceWithInfo**, which simply allows us to pass the video frame and the system time of the frame - both required by the **MediaStreamSource** - as a single object. The first step of the frame copy process is to instantiate this class and set the system time.

The next steps are the part of this example that relies specifically on the SharpDX library. The helper functions used here are defined at the end of this article. First we use the **MultiThreadLock** to make sure no other threads access the video frame buffer while we are making the copy. Next, we call the helper method **CreateSharpDXTexture2D** to create a SharpDX **Texture2D** object from the video frame. This will be the source texture for the copy operation.

Next, we copy from the **Texture2D** object created in the previous step into the composition texture we created earlier in the process. This composition texture acts as a swap buffer so that the encoding process can operate on the pixels while the next frame is being captured. To perform the copy, we clear the render target view associated with the composition texture, then we define the region within the texture we want to copy - the entire texture in this case, and then we call **CopySubresourceRegion** to actually copy the pixels to the composition texture.

We create a copy of the texture description to use when we create our target texture, but the description is modified, setting the **BindFlags** to **RenderTarget** so that the new texture has write access. Setting the **CpuAccessFlags** to **None** allows the system to optimize the copy operation. The texture description is used to create a new texture resource and the composition texture resource is copied into this new resource with a call to **CopyResource**. Finally, **CreateDirect3DSurfaceFromSharpDXTexture** is called to create the **IDirect3DSurface** object that is returned from this method.

C#

```
public SurfaceWithInfo WaitForNewFrame()
{
    // Let's get a fresh one.
    _currentFrame?.Dispose();
    _frameEvent.Reset();

    var signaledEvent = _events[WaitHandle.WaitAny(_events)];
    if (signaledEvent == _closedEvent)
    {
        Cleanup();
        return null;
    }

    var result = new SurfaceWithInfo();
    result.SystemRelativeTime = _currentFrame.SystemRelativeTime;
    using (var multithreadLock = new MultithreadLock(_multithread))
    using (var sourceTexture =
Direct3D11Helpers.CreateSharpDXTexture2D(_currentFrame.Surface))
    {

        _sharpDxD3dDevice.ImmediateContext.ClearRenderTargetView(_composeRenderTarge
tView, new SharpDX.Mathematics.Interop.RawColor4(0, 0, 0, 1));

        var width = Math.Clamp(_currentFrame.ContentSize.Width, 0,
        _currentFrame.Surface.Description.Width);
        var height = Math.Clamp(_currentFrame.ContentSize.Height, 0,
        _currentFrame.Surface.Description.Height);
        var region = new SharpDX.Direct3D11.ResourceRegion(0, 0, 0, width,
        height, 1);

        _sharpDxD3dDevice.ImmediateContext.CopySubresourceRegion(sourceTexture, 0,
        region, _composeTexture, 0);

        var description = sourceTexture.Description;
        description.Usage = SharpDX.Direct3D11.ResourceUsage.Default;
        description.BindFlags = SharpDX.Direct3D11.BindFlags.ShaderResource
| SharpDX.Direct3D11.BindFlags.RenderTarget;
        description.CpuAccessFlags = SharpDX.Direct3D11.CpuAccessFlags.None;
        description.OptionFlags =
SharpDX.Direct3D11.ResourceOptionFlags.None;

        using (var copyTexture = new
SharpDX.Direct3D11.Texture2D(_sharpDxD3dDevice, description))
        {
            _sharpDxD3dDevice.ImmediateContext.CopyResource(_composeTexture,
            copyTexture);
            result.Surface =
Direct3D11Helpers.CreateDirect3DSurfaceFromSharpDXTexture(copyTexture);
        }
    }
}
```

```
    return result;
}
```

Stop capture and clean up resources

The **Stop** method provides a way to stop the capture operation. Your app may call this programmatically or in response to a user interaction, like a button click. This method simply sets the **_closedEvent**. The **WaitForNewFrame** method defined in the previous steps looks for this event and, if set, shuts down the capture operation.

C#

```
private void Stop()
{
    _closedEvent.Set();
}
```

The **Cleanup** method is used to properly dispose of the resources that were created during the copy operation. This includes:

- The **Direct3D11CaptureFramePool** object used by the capture session
- The **GraphicsCaptureSession** and **GraphicsCaptureItem**
- The Direct3D and SharpDX devices
- The SharpDX texture and render target view used in the copy operation.
- The **Direct3D11CaptureFrame** used for storing the current frame.

C#

```
private void Cleanup()
{
    _framePool?.Dispose();
    _session?.Dispose();
    if (_captureItem != null)
    {
        _captureItem.Closed -= OnClosed;
    }
    _captureItem = null;
    _device = null;
    _sharpDxD3dDevice = null;
    _composeTexture?.Dispose();
    _composeTexture = null;
    _composeRenderTargetView?.Dispose();
    _composeRenderTargetView = null;
    _currentFrame?.Dispose();
}
```

Helper wrapper classes

The following helper classes were defined to help with the example code in this article.

The **MultithreadLock** helper class wraps the SharpDX **Multithread** class that makes sure that other threads don't access the texture resources while being copied.

C#

```
class MultithreadLock : IDisposable
{
    public MultithreadLock(SharpDX.Direct3D11.Multithread multithread)
    {
        _multithread = multithread;
        _multithread?.Enter();
    }

    public void Dispose()
    {
        _multithread?.Leave();
        _multithread = null;
    }

    private SharpDX.Direct3D11.Multithread _multithread;
}
```

SurfaceWithInfo is used to associate an **IDirect3DSurface** with a **SystemRelativeTime** representing a captured frame and the time it was captured, respectively.

C#

```
public sealed class SurfaceWithInfo : IDisposable
{
    public IDirect3DSurface Surface { get; internal set; }
    public TimeSpan SystemRelativeTime { get; internal set; }

    public void Dispose()
    {
        Surface?.Dispose();
        Surface = null;
    }
}
```

Direct3D and SharpDX helper APIs

The following helper APIs are defined to abstract out the creation of Direct3D and SharpDX resources. A detailed explanation of these technologies is outside the scope of

this article but the code is provided here to allow you to implement the example code shown in the walkthrough.

C#

```
[ComImport]
[Guid("A9B3D012-3DF2-4EE3-B8D1-8695F457D3C1")]
[InterfaceType(ComInterfaceType.InterfaceIsIUnknown)]
[ComVisible(true)]
interface IDirect3DDxgiInterfaceAccess
{
    IntPtr GetInterface([In] ref Guid iid);
};

public static class Direct3D11Helpers
{
    internal static Guid IInspectable = new Guid("AF86E2E0-B12D-4c6a-9C5A-D7AA65101E90");
    internal static Guid ID3D11Resource = new Guid("dc8e63f3-d12b-4952-b47b-5e45026a862d");
    internal static Guid IDXGIAdapter3 = new Guid("645967A4-1392-4310-A798-8053CE3E93FD");
    internal static Guid ID3D11Device = new Guid("db6f6ddb-ac77-4e88-8253-819df9bbf140");
    internal static Guid ID3D11Texture2D = new Guid("6f15aaf2-d208-4e89-9ab4-489535d34f9c");

    [DllImport(
        "d3d11.dll",
        EntryPoint = "CreateDirect3D11DeviceFromDXGIDevice",
        SetLastError = true,
        CharSet = CharSet.Unicode,
        ExactSpelling = true,
        CallingConvention = CallingConvention.StdCall
    )]
    internal static extern UInt32
    CreateDirect3D11DeviceFromDXGIDevice(IntPtr dxgiDevice, out IntPtr
    graphicsDevice);

    [DllImport(
        "d3d11.dll",
        EntryPoint = "CreateDirect3D11SurfaceFromDXGISurface",
        SetLastError = true,
        CharSet = CharSet.Unicode,
        ExactSpelling = true,
        CallingConvention = CallingConvention.StdCall
    )]
    internal static extern UInt32
    CreateDirect3D11SurfaceFromDXGISurface(IntPtr dxgiSurface, out IntPtr
    graphicsSurface);

    public static IDirect3DDevice Created3DDevice()
    {
        return Created3DDevice(false);
    }
}
```

```

    }

    public static IDirect3DDevice CreateD3DDevice(bool useWARP)
    {
        var d3dDevice = new SharpDX.Direct3D11.Device(
            useWARP ? SharpDX.Direct3D.DriverType.Software :
            SharpDX.Direct3D.DriverType.Hardware,
            SharpDX.Direct3D11.DeviceCreationFlags.BgraSupport);
        IDirect3DDevice device = null;

        // Acquire the DXGI interface for the Direct3D device.
        using (var dxgiDevice =
            d3dDevice.QueryInterface<SharpDX.DXGI.Device3>())
        {
            // Wrap the native device using a WinRT interop object.
            uint hr =
                CreateDirect3D11DeviceFromDXGIDevice(dxgiDevice.NativePointer, out IntPtr
                pUnknown);

            if (hr == 0)
            {
                device = Marshal.GetObjectForIUnknown(pUnknown) as
                IDirect3DDevice;
                Marshal.Release(pUnknown);
            }
        }

        return device;
    }

    internal static IDirect3DSurface
    CreateDirect3DSurfaceFromSharpDXTexture(SharpDX.Direct3D11.Texture2D
    texture)
    {
        IDirect3DSurface surface = null;

        // Acquire the DXGI interface for the Direct3D surface.
        using (var dxgiSurface =
            texture.QueryInterface<SharpDX.DXGI.Surface>())
        {
            // Wrap the native device using a WinRT interop object.
            uint hr =
                CreateDirect3D11SurfaceFromDXGISurface(dxgiSurface.NativePointer, out IntPtr
                pUnknown);

            if (hr == 0)
            {
                surface = Marshal.GetObjectForIUnknown(pUnknown) as
                IDirect3DSurface;
                Marshal.Release(pUnknown);
            }
        }

        return surface;
    }

```

```

    }

    internal static SharpDX.Direct3D11.Device
    CreateSharpDXDevice(IDirect3DDevice device)
    {
        var access = (IDirect3DXgiInterfaceAccess)device;
        var d3dPointer = access.GetInterface(ID3D11Device);
        var d3dDevice = new SharpDX.Direct3D11.Device(d3dPointer);
        return d3dDevice;
    }

    internal static SharpDX.Direct3D11.Texture2D
    CreateSharpDXTexture2D(IDirect3DSurface surface)
    {
        var access = (IDirect3DXgiInterfaceAccess)surface;
        var d3dPointer = access.GetInterface(ID3D11Texture2D);
        var d3dSurface = new SharpDX.Direct3D11.Texture2D(d3dPointer);
        return d3dSurface;
    }

    public static SharpDX.Direct3D11.Texture2D InitializeComposeTexture(
        SharpDX.Direct3D11.Device sharpDxD3dDevice,
        SizeInt32 size)
    {
        var description = new SharpDX.Direct3D11.Texture2DDescription
        {
            Width = size.Width,
            Height = size.Height,
            MipLevels = 1,
            ArraySize = 1,
            Format = SharpDX.DXGI.Format.B8G8R8A8_UNorm,
            SampleDescription = new SharpDX.DXGI.SampleDescription()
            {
                Count = 1,
                Quality = 0
            },
            Usage = SharpDX.Direct3D11.ResourceUsage.Default,
            BindFlags = SharpDX.Direct3D11.BindFlags.ShaderResource |
            SharpDX.Direct3D11.BindFlags.RenderTarget,
            CpuAccessFlags = SharpDX.Direct3D11.CpuAccessFlags.None,
            OptionFlags = SharpDX.Direct3D11.ResourceOptionFlags.None
        };
        var composeTexture = new
        SharpDX.Direct3D11.Texture2D(sharpDxD3dDevice, description);

        using (var renderTargetView = new
        SharpDX.Direct3D11.RenderTargetView(sharpDxD3dDevice, composeTexture))
        {
            sharpDxD3dDevice.ImmediateContext.ClearRenderTargetView(renderTargetView,
            new SharpDX.Mathematics.Interop.RawColor4(0, 0, 0, 1));
        }
    }

```

```
    }  
    return composeTexture;  
  }  
}
```

See also

- [Windows.Graphics.Capture Namespace](#)
- [Screen capture](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

App-to-app communication

Article • 10/20/2022

This section explains how to share data between Universal Windows Platform (UWP) apps, including how to use the Share contract, copy and paste, drag and drop, and app services.

The Share contract is one way users can quickly exchange data between apps. For example, a user might want to share a webpage with their friends using a social networking app, or save a link in a notes app to refer to later. Consider using a Share contract if your app receives content in scenarios that a user can quickly complete while in the context of another app.

An app can support the Share feature in two ways. First, it can be a [source app that provides content that the user wants to share](#). Second, the app can be a [target app that the user selects as the destination for shared content](#). An app can also be both a source app and a target app. If you want your app to share content as a source app, you need to decide what data formats your app can provide.

In addition to the Share contract, apps can also integrate classic techniques for transferring data, such as [dragging and dropping](#) or [copy and pasting](#). In addition to communication between UWP apps, these methods also support sharing to and from desktop applications.

UWP apps can also create [app services](#) that provide functionality to other UWP apps. An app service runs as a background task in the host app and can provide its service to other apps. For example, an app service might provide a bar code scanner service that other apps could use.

In this section

Topic	Description
Share data	This article explains how to support the Share contract in a UWP app. The Share contract is an easy way to quickly share data, such as text, links, photos, and videos, between apps. For example, a user might want to share a webpage with their friends using a social networking app, or save a link in a notes app to refer to later.
Receive data	This article explains how to receive content in your UWP app shared from another app using Share contract. This Share contract allows your app to be presented as an option when the user invokes Share.

Topic	Description
Copy and paste	This article explains how to support copy and paste in UWP apps using the clipboard. Copy and paste is the classic way to exchange data either between apps, or within an app, and almost every app can support clipboard operations to some degree.
Drag and drop	This article explains how to add dragging and dropping in your UWP app. Drag and drop is a classic, natural way of interacting with content such as images and files. Once implemented, drag and drop works seamlessly in all directions, including app-to-app, app-to-desktop, and desktop-to app.
Create and consume an app service	This article explains how to create an app service in a UWP app that provides services to other UWP apps.

See also

- [Develop UWP apps](#)

Share data

Article • 10/18/2023

This article explains how to support the Share contract in a desktop or a Universal Windows Platform (UWP) app. The Share contract is an easy way to quickly share data—such as text, links, photos, and videos—between apps. For example, a user might want to share a webpage with their friends using a social networking app, or save a link in a notes app to refer to later.

ⓘ Note

The code examples in this article are from UWP apps. Desktop apps should use the `IDataTransferManagerInterop`. For more info, and code examples, see [Display WinRT UI objects that depend on CoreWindow](#). Also see the [WPF Sharing content source app sample](#) ↗.

Set up an event handler

Add a [DataRequested](#) event handler to be called whenever a user invokes share. This can occur either when the user taps a control in your app (such as a button or app bar command) or automatically in a specific scenario (if the user finishes a level and gets a high score, for example).

C#

```
DataTransferManager dataTransferManager =  
DataTransferManager.GetForCurrentView();  
dataTransferManager.DataRequested += DataTransferManager_DataRequested;
```

When a [DataRequested](#) event occurs, your app receives a [DataRequest](#) object. This contains a [DataPackage](#) that you can use to provide the content that the user wants to share. You must provide a title and data to share. A description is optional, but recommended.

C#

```
DataRequest request = args.Request;
```

Choose data

You can share various types of data, including:

- Plain text
- Uniform Resource Identifiers (URIs)
- HTML
- Formatted text
- Bitmaps
- Files
- Custom developer-defined data

The [DataPackage](#) object can contain one or more of these formats, in any combination. The following example demonstrates sharing text.

C#

```
request.Data.SetText("Hello world!");
```

Set properties

When you package data for sharing, you can supply a variety of properties that provide additional information about the content being shared. These properties help target apps improve the user experience. For example, a description helps when the user is sharing content with more than one app. Adding a thumbnail when sharing an image or a link to a web page provides a visual reference to the user. For more information, see [DataPackagePropertySet](#).

Warning

All properties except the title are optional. The title property is mandatory and must be set.

C#

```
request.Data.Properties.Title = "Share Example";  
request.Data.Properties.Description = "A demonstration on how to share";
```

Launch the share UI

A UI for sharing is provided by the system. To launch it, call the [ShowShareUI](#) method.

C#

```
DataManager.ShowShareUI();
```

Handle errors

In most cases, sharing content is a straightforward process. However, there's always a chance that something unexpected could happen. For example, the app might require the user to select content for sharing but the user didn't select any. To handle these situations, use the [FailWithDisplayText](#) method, which will display a message to the user if something goes wrong.

Delay share with delegates

Sometimes, it might not make sense to prepare the data that the user wants to share right away. For example, if your app supports sending a large image file in several different possible formats, it's inefficient to create all those images before the user makes their selection.

To solve this problem, a [DataPackage](#) can contain a delegate — a function that is called when the receiving app requests data. We recommend using a delegate any time that the data a user wants to share is resource-intensive.

C#

```
async void OnDeferredImageRequestedHandler(DataProviderRequest request)
{
    // Provide updated bitmap data using delayed rendering
    if (this.imageStream != null)
    {
        DataProviderDeferral deferral = request.GetDeferral();
        InMemoryRandomAccessStream inMemoryStream = new
        InMemoryRandomAccessStream();

        // Decode the image.
        BitmapDecoder imageDecoder = await
        BitmapDecoder.CreateAsync(this.imageStream);

        // Re-encode the image at 50% width and height.
        BitmapEncoder imageEncoder = await
        BitmapEncoder.CreateForTranscodingAsync(inMemoryStream, imageDecoder);
        imageEncoder.BitmapTransform.ScaledWidth = (uint)
        (imageDecoder.OrientedPixelWidth * 0.5);
        imageEncoder.BitmapTransform.ScaledHeight = (uint)
        (imageDecoder.OrientedPixelHeight * 0.5);
        await imageEncoder.FlushAsync();
    }
}
```

```
request.SetData(RandomAccessStreamReference.CreateFromStream(inMemoryStream)
);
    deferral.Complete();
}
}
```

See also

- [Display WinRT UI objects that depend on CoreWindow](#)
- [App-to-app communication](#)
- [Receive data](#)
- [DataPackage](#)
- [DataPackagePropertySet](#)
- [DataRequest](#)
- [DataRequested](#)
- [FailWithDisplayText](#)
- [ShowShareUi](#)

Receive data

Article • 10/20/2022

This article explains how to receive content in your Universal Windows Platform (UWP) app shared from another app by using Share contract. This Share contract allows your app to be presented as an option when the user invokes Share.

Declare your app as a share target

The system displays a list of possible target apps when a user invokes Share. In order to appear on the list, your app needs to declare that it supports the Share contract. This lets the system know that your app is available to receive content.

1. Open the manifest file. It should be called something like **package.appxmanifest**.
2. Open the **Declarations** tab.
3. Choose **Share Target** from the **Available Declarations** list, and then select **Add**.

Choose file types and formats

Next, decide what file types and data formats you support. The Share APIs support several standard formats, such as Text, HTML, and Bitmap. You can also specify custom file types and data formats. If you do, remember that source apps have to know what those types and formats are; otherwise, those apps can't use the formats to share data.

Only register for formats that your app can handle. Only target apps that support the data being shared appear when the user invokes Share.

To set file types:

1. Open the manifest file. It should be called something like **package.appxmanifest**.
2. In the **Supported File Types** section of the **Declarations** page, select **Add New**.
3. Type the file name extension that you want to support, for example, ".docx." You need to include the period. If you want to support all file types, select the **SupportsAnyFileType** check box.

To set data formats:

1. Open the manifest file.
2. Open the **Data Formats** section of the **Declarations** page, and then select **Add New**.
3. Type the name of the data format you support, for example, "Text."

Handle share activation

When a user selects your app (usually by selecting it from a list of available target apps in the share UI), an [OnShareTargetActivated](#) event is raised. Your app needs to handle this event to process the data that the user wants to share.

C#

```
protected override async void
OnShareTargetActivated(ShareTargetActivatedEventArgs args)
{
    // Code to handle activation goes here.
}
```

The data that the user wants to share is contained in a [ShareOperation](#) object. You can use this object to check the format of the data it contains.

C#

```
ShareOperation shareOperation = args.ShareOperation;
if (shareOperation.Data.Contains(StandardDataFormats.Text))
{
    string text = await shareOperation.Data.GetTextAsync();

    // To output the text from this example, you need a TextBlock control
    // with a name of "sharedContent".
    sharedContent.Text = "Text: " + text;
}
```

Report sharing status

In some cases, it can take time for your app to process the data it wants to share. Examples include users sharing collections of files or images. These items are larger than a simple text string, so they take longer to process.

C#

```
shareOperation.ReportStarted();
```

After calling [ReportStarted](#), don't expect any more user interaction with your app. As a result, you shouldn't call it unless your app is at a point where it can be dismissed by the user.

With an extended share, it's possible that the user might dismiss the source app before your app has all the data from the `DataPackage` object. As a result, we recommend that you let the system know when your app has acquired the data it needs. This way, the system can suspend or terminate the source app as necessary.

C#

```
shareOperation.ReportSubmittedBackgroundTask();
```

If something goes wrong, call [ReportError](#) to send an error message to the system. The user will see the message when they check on the status of the share. At that point, your app is shut down and the share is ended. The user will need to start again to share the content to your app. Depending on your scenario, you may decide that a particular error isn't serious enough to end the share operation. In that case, you can choose to not call **ReportError** and to continue with the share.

C#

```
shareOperation.ReportError("Could not reach the server! Try again later.");
```

Finally, when your app has successfully processed the shared content, you should call [ReportCompleted](#) to let the system know.

C#

```
shareOperation.ReportCompleted();
```

When you use these methods, you usually call them in the order just described, and you don't call them more than once. However, there are times when a target app can call [ReportDataRetrieved](#) before [ReportStarted](#). For example, the app might retrieve the data as part of a task in the activation handler, but not call **ReportStarted** until the user selects a **Share** button.

Return a QuickLink if sharing was successful

When a user selects your app to receive content, we recommend that you create a [QuickLink](#). A **QuickLink** is like a shortcut that makes it easier for users to share information with your app. For example, you could create a **QuickLink** that opens a new mail message pre-configured with a friend's email address.

A **QuickLink** must have a title, an icon, and an Id. The title (like "Email Mom") and icon appear when the user taps the Share charm. The Id is what your app uses to access any